Birds AI Sound Classifier - Prozess- & Überwachungskonzept (Version V1.0.0)

Dokumentation: Zusammenspiel von 'autostart_monitor.py', 'start.bat' und 'birds_ai_sound_classifier.py'
Datum: Oktober 2026
Status: Freigegeben

---

1. Übersicht & Rollenverteilung

'autostart_monitor.py' und 'start.bat' bilden zusammen ein aufeinander abgestimmtes Gespann zur zuverlässigen Ausführung und Überwachung der Anwendung 'birds_ai_sound_classifier.py'.

* 'autostart_monitor.py' (Watchdog / Supervisor):
Übernimmt die kontinuierliche Prozessüberwachung, die Erkennung und Bereinigung doppelter Instanzen, das Aufräumen verwaister Hintergrundprozesse sowie den automatischen Neustart bei Ausfall.
* 'start.bat' (Starter / Launcher):
Stellt das korrekte Ausführungsverzeichnis ('%~dp0') sicher und startet die Python-Hauptanwendung in einem eigenständigen Konsolenfenster.
* 'birds_ai_sound_classifier.py' (Hauptanwendung):
Führt die Webserver-Dienste (Flask / Waitress) sowie den Audio-Erkennungsdienst aus (ausgelagert in einen separaten Multiprocessing-Kindprozess).

Komponente	Rolle	Hauptaufgabe
'autostart_monitor.py' / '.exe'	Supervisor / Wächter	Statusprüfung alle 60s, Entprellen, Killen von Duplikaten/Zombies, Neustart
'start.bat'	Launcher / Wrapper	Arbeitsverzeichnis setzen, Konsolenaufruf mit Pause bei Fehlern
'birds_ai_sound_classifier.py'	Hauptanwendung	Flask/Waitress-Webserver, mDNS & Audioanalyse-Multiprocessing

---

2. Startprozedur & Ausführungskontext

Der Startvorgang wird über die Methode 'start_app()' gesteuert:

1. Bevorzugter Aufruf:
Der Monitor prüft primär auf das Vorhandensein der Datei 'start.bat' im Anwendungsverzeichnis.
2. Eigenes Konsolenfenster:
Wenn vorhanden, startet er 'start.bat' mit dem Flag 'subprocess.CREATE_NEW_CONSOLE'. Dadurch erhält die Anwendung ihr eigenes Konsolenfenster für Standardausgaben und Fehlermeldungen, getrennt vom Monitorfenster.
3. Arbeitsverzeichnis-Integrität:
'start.bat' wechselt mit 'cd /d "%~dp0"' zwingend in das Verzeichnis der Skripte, sodass relative Pfade (z. B. zu Modellen, Datenbanken und Konfigurationsdateien) korrekt aufgelöst werden.
4. Fallback:
Sollte 'start.bat' nicht existieren, greift der Monitor auf einen direkten Aufruf zurück ('subprocess.Popen(["python", TARGET_APP], ...)').

---

3. Erkennung doppelter Instanzen (Duplicate Detection)

Die Erkennung laufender Instanzen erfolgt über 'get_running_instances()' und wird periodisch im 'monitor_loop()' ausgeführt:

* Filterkriterien:
Alle Systemprozesse werden mit 'psutil' nach Prozessen durchsucht, deren Name 'python' enthält und in deren Befehlszeile ('cmdline') der Name 'birds_ai_sound_classifier.py' vorkommt.
* Keine Fehlalarme durch Multiprocessing-Kinder:
Unter Windows erzeugt Pythons 'multiprocessing'-Modul Kindprozesse über 'multiprocessing.spawn'. Der Dateiname 'birds_ai_sound_classifier.py' taucht dabei üblicherweise nicht als Direktbefehl im 'cmdline'-String des Kindprozesses auf. Der Audio-Kindprozess wird daher richtigerweise nicht als eigenständige zweite Hauptinstanz gewertet.
* Priorisierung nach Alter (FIFO-Prinzip):
Werden mehrere Instanzen festgestellt ('len(instances) > 1'), sortiert der Monitor die Liste chronologisch nach 'create_time':
* Die älteste Instanz ('instances[0]') bleibt unberührt im laufenden Betrieb.
* Alle überzähligen, neueren Instanzen ('instances[1:]') werden mitsamt ihrem Prozessbaum beendet.

---

4. Prozess-Terminierung (Main- & Childprozesse)

Sowohl beim Bereinigen von Duplikaten als auch beim geordneten Beenden setzt die Funktion 'kill_process_tree()' eine 5-stufige Kaskade ein:

1. CMD-Fenster merken:
Ermittelt den übergeordneten Elternprozess ('proc.parent()'). Wenn dieser 'cmd.exe' ist (bedingt durch den Start via 'start.bat'), wird er für das spätere Schließen vorgemerkt.
2. Kindprozesse erfassen:
Über 'proc.children(recursive=True)' werden alle untergeordneten Prozesse (wie z. B. der Audioanalyse-Subprozess) ermittelt.
3. Sanftes Beenden ('terminate'):
Zuerst wird an alle Kindprozesse und unmittelbar danach an den Hauptprozess ein 'terminate()' gesendet.
4. Force-Kill mit Timeout:
Über 'psutil.wait_procs(..., timeout=3)' wird bis zu 3 Sekunden gewartet. Prozesse, die nach Ablauf des Timeouts noch aktiv sind, werden hart mit 'kill()' beendet.
5. Konsolenfenster bereinigen:
Abschließend wird der gemerkte 'cmd.exe'-Prozess beendet, sodass keine verwaisten leeren Eingabeaufforderungen auf dem Desktop verbleiben.

---

5. Bereinigung verwaister Hintergrundprozesse

Stürzt die Hauptanwendung unerwartet ab ('len(instances) == 0'), greift vor dem Neustart die Funktion 'cleanup_orphaned_processes()':

* Sie durchsucht das System nach verbliebenen Python-Prozessen im Arbeitsverzeichnis der Anwendung.
* Prozesse, die 'multiprocessing.spawn' oder 'birds_ai_sound_classifier.py' in der Aufrufzeile tragen, werden identifiziert und beendet.
* Nutzen: Es wird verhindert, dass blockierte Audio-Streams (PyAudio/Mikrofon) oder gesperrte SQLite-Datenbanken einen sauberen Neustart verhindern.
* Selbstschutz: Der Monitor selbst schließt sich über eine Namensprüfung ('"autostart_monitor" in cmd_str') von der

Bereinigung aus.

---

6. Wichtige Betriebs- und Praxishinweise

* Prüfintervall (60 Sekunden):

Die Überwachungsschleife prüft im 60-Sekunden-Takt. Wird die Anwendung versehentlich manuell ein zweites Mal gestartet, läuft das Duplikat maximal 60 Sekunden, bis der Monitor den Takt erreicht und die überzählige Instanz eliminiert.

* Verhalten bei Fehlerabbruch ('pause' in 'start.bat'):

In 'start.bat' bewirkt die Zeile 'if %errorlevel% neq 0 pause', dass bei einem Crash die Konsole geöffnet bleibt, um die Fehlerausgabe lesen zu können.

Effekt: Der Python-Prozess existiert nicht mehr ('len(instances) == 0'). Der Monitor startet daraufhin die App in einem neuen Konsolenfenster. Das alte Konsolenfenster mit der Fehlermeldung bleibt zur Analyse offen stehen und muss anschließend manuell geschlossen werden.

* Empfohlene Autostart-Einbindung:

In den Windows-Autostart ('shell:startup') gehört ausschließlich 'autostart_monitor.exe' (bzw. 'autostart_monitor.py'), nicht 'start.bat'. Der Monitor übernimmt den eigentlichen Anwendungsstart vollautomatisch.

* Zusammenspiel mit GitHub Software-Updates (Manuell & Auto-Update 02:00 Uhr):

Wird über die Weboberfläche ('settings.html') ein manuelles Software-Update ausgelöst oder durch den aktivierten Hintergrund-Scheduler um 02:00 Uhr nachts vollautomatisch ein neues GitHub-Release installiert, beendet sich 'birds_ai_sound_classifier.py' nach erfolgreichem Dateiaustausch samt zugehörigem 'cmd.exe'-Fenster selbstständig. 'autostart_monitor.py' erkennt dies beim nächsten Prüfzyklus (innerhalb von max. 60 Sekunden) und startet die aktualisierte Anwendung automatisch neu.